HCMC UNIVERSITY OF TECHNOLOGY AND ENGINEERING

FACULTY OF INFORMATION TECHNOLOGY



FINAL TERM PROJECT

Course name: Digital Image Processing (DIP)

REAL-TIME AMERICAN SIGN LANGUAGE RECOGNITION: FROM HAND LANDMARKS TO WORD SEQUENCES AND SENTENCE COMPOSITION

Lecturer name: Assoc. Prof. Hoang Van Dung

List of members

Student ID	Student name	Contribution (%)
24110086	Tô Phương Đạt	100%
24110120	Trương Khắc Nhật Phi	100%

Ho Chi Minh City, 10/2026

ABSTRACT

This report presents a real-time American Sign Language (ASL) recognition system that runs on a CPU with an ordinary webcam. MediaPipe hand landmarks are normalised and cut into strokes by a velocity-based state machine; each stroke becomes a 24-dimensional trajectory that is matched by Dynamic Time Warping (DTW) against a small bank of templates, with a three-level decision (accepted, uncertain, rejected) that refuses unknown input. A small spotter network, a one-shot lock and a landmark hand-shape gate suppress everyday movements, and a rule-based composer turns the word sequence into English and Vietnamese sentences. New words and new signers are added from one to three samples, without training.

On the owner's bank the leave-one-session-out accuracy is 100%; for unseen signers the forced-choice top-1 accuracy is 92.2% and none of 175 unknown words is accepted. Checks made after the first measurements (Section 4.11) found a mirroring defect that disabled the spotter on the live camera, showed that over-exposed images make recognition collapse, and measured the five added words. The data come mainly from one signer, so every figure is optimistic for other people.

TABLE CONTENTS

PART 1. PROJECT DESCRIPTION 0
1.1. Problem statement 0
1.2. Objectives and requirements 0
1.3. Scope and subjects 1
1.4. Structure of the report 1
PART 2. BACKGROUND KNOWLEDGE 2
2.1. Related solutions 2
2.2. Tools and development environment 2
2.3. Hand landmarks 3
2.4. Digital image processing techniques used 3
2.5. Dynamic Time Warping 4
2.6. Evaluation protocols and metrics 4
PART 3. ANALYSIS AND SOLUTION DESIGN 5
3.1. System architecture 5
3.2. Design decisions at a glance: what was replaced and why 6
3.3. Perception and normalisation (Layer 1) 7
3.4. Segmentation and anti-false-trigger (Layer 2) 7
3.5. Trajectory features (Layer 3, part 1) 9
3.6. Matching and the three-level decision (Layer 3, part 2) 9
3.7. Auxiliary layers 10
3.8. Enrolment and confirmation 11
3.9. Fingerspelling 11
3.10. Sentence composition and output (Layer 4) 11
PART 4. EXPERIMENTS, EVALUATION AND DISCUSSION 12
4.1. Experimental setup 12
4.2. Word recognition: DTW against learned baselines 12
4.3. Adapting to a new signer 14
4.4. Gated deep-learning fallback 17
4.5. Robustness of segmentation to camera frame rate 17
4.6. Hand shape and fingerspelling 18
4.7. Why the current methods replaced the earlier ones 18
4.8. Automated software testing 23
4.9. User interface and how to run the software 23
4.10. Discussion and limitations 27
4.11. Additional experiments after the first measurements 28
PART 5. CONCLUSIONS 33
5.1. Results achieved 33
5.2. Honest assessment 33
5.3. Future work 33
REFERENCES 34
APPENDIX A. THE PIPELINE OVERVIEW FILE AND THE SUBMITTED PACKAGE 35
A.1. pipeline.py: the main workflow in one short file 35
A.2. Submitted package 36
APPENDIX B. MAIN SOURCE FILES 37

ABBREVIATION LIST

Abbreviation	Meaning
ASL	American Sign Language
AUROC	Area Under the Receiver Operating Characteristic curve
CLAHE	Contrast Limited Adaptive Histogram Equalization
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DIP	Digital Image Processing
DL	Deep Learning
DTW	Dynamic Time Warping
EMA	Exponential Moving Average
FPS	Frames Per Second
FSM	Finite-State Machine
GRU	Gated Recurrent Unit
LOSO	Leave-One-Session-Out
LOWO	Leave-One-Word-Out
MLP	Multi-Layer Perceptron
1-NN	1-Nearest Neighbour
SAPI	Speech Application Programming Interface
TTS	Text-To-Speech
UI	User Interface

FIGURE LIST

Figure 1 – Real-time pipeline from webcam frame to spoken sentence 5
Figure 2 – Unseen-signer accuracy and wrongly accepted unknown words for four recognisers 13
Figure 3 – LOSO top-1 against the number of reference samples per word 14
Figure 4 – Effect of enrolling a stranger's own samples compared with pooling other signers 15
Figure 5 – Correct-accepted rate with and without confirmation questions in one session 16
Figure 6 – Unseen-signer accuracy of DTW, deep learning and the gated hybrid 17
Figure 7 – Share of strokes segmented correctly at different frame rates 18
Figure 8 – Ablation of feature groups on the ASL Citizen bank (139 samples, 19 words) 20
Figure 9 – Main window of the application in its idle state: camera view (left), recognition monitor (right) and sentence panel with key legend (bottom) 24
Figure 10 – Main window during live recognition: hand skeleton with per-finger labels (left), recognition monitor with the candidate list, the activity log and the spotter indicator (right), and the sentence panel (bottom) 25
Figure 11 – The DIP 5-step pipeline inspector (key d): the image-processing chain applied to the hand crop, from the filtered crop to the Sobel gradient, and the three-channel tensor it produces 26
Figure 12 – Frame AUC of the spotter on original, mirrored and perturbed input, with and without augmentation 29
Figure 13 – Correct, missed and stray words when the recording is degraded before replay 30
Figure 14 – Top-1 accuracy of the five added words (leave-block-out) 31
Figure 15 – The eight processing steps of pipeline.py 35

TABLE LIST

Table 1 – Functional and non-functional requirements....0
Table 2 – Scope of the current version....1
Table 3 – Families of sign recognition solutions....2
Table 4 – Software stack....2
Table 5 – DIP techniques and where they are used....3
Table 6 – Methods replaced or rejected, and the evidence....6
Table 7 – Feature groups of the 24-D vector and weights used in production (G1p90.json)....9
Table 8 – Decision rules....10
Table 9 – Examples of gloss-to-sentence rules....11
Table 10 – Data used in the experiments....12
Table 11 – Closed-set accuracy and unknown-word discrimination....13
Table 12 – Operating point at equal LOSO acceptance (152 correct)....13
Table 13 – Correct and accepted (%) when k own samples per word are added (5 repetitions)....15
Table 14 – Segmentation with frame-count thresholds against time_norm....17
Table 15 – Per-letter accuracy of the image CNN on all stored images (including training images)....19
Table 16 – Test of ASL-LEX phonological weights on 10 words (68 Citizen samples)....20
Table 17 – Threshold strategies on the full bank (LOSO 175, Mai 51, LOWO 175, negatives 41)....21
Table 18 – Effect of adding synthetic variants (7 per stroke) to the bank....22
Table 19 – Main controls....26
Table 20 – Spotter trained on originals versus trained with augmentation (out-of-fold)....29
Table 21 – Replay of one 29-word session under image degradations (default configuration)....30
Table 22 – Leave-block-out results of the five added words (18-word bank)....31
Table 23 – Rejection scores compared (AUROC and recall of correct queries at 5% false acceptance)....32
Table 24 – Steps of pipeline.py, the module that implements each and the method applied....35
Table 25 – Main source files and their role....37

PART 1. PROJECT DESCRIPTION

1.1. Problem statement

American Sign Language (ASL) is a visual language in which meaning is carried by hand shape, location, orientation and movement. A system that converts signing into text and speech can lower the communication barrier between Deaf and hearing people, but it is only useful if it runs on equipment people already own. This project builds such a system for an ordinary laptop: a webcam is the only sensor, a CPU is the only processor, and the output is a recognised word, a composed sentence and a spoken voice, all in real time.

The problem is harder than classifying a still picture of a hand. Four difficulties shaped the whole design:

- **Signer variability.** Hand size, distance to the camera, handedness, signing speed and personal style differ from person to person. A model that works for its owner often fails on a stranger (Section 4.3).
- **Continuous input.** The camera never stops. The system must decide when a sign starts and ends, ignore everyday movements such as scratching the face or drinking water, and refuse words that are not in its vocabulary.
- **Very little data.** Only a handful of volunteers recorded a few dozen samples per word, which rules out training a large video network from scratch.
- **Real-time and CPU-only.** The live loop must sustain at least 25 frames per second, and speech output must never block the camera loop.

1.2. Objectives and requirements

The project is a Digital Image Processing (DIP) course project. Its objective is to design, implement and evaluate a pipeline that recognises ASL words from a webcam stream and assembles them into sentences. The requirements are listed in Table 1.

Table 1 – Functional and non-functional requirements

No.	Requirement	Description
F1	Isolated word recognition	Recognise the production vocabulary (13 words) and the demonstration vocabulary (18 words) from short hand movements.
F2	Rejection	Output ACCEPTED, UNCERTAIN or REJECTED; unknown words and everyday movements must not be forced into a vocabulary word.
F3	Sentence composition	Collect words into a sentence and turn the gloss sequence into a natural English/Vietnamese sentence using offline rules.

No.	Requirement	Description
F4	Fingerspelling	Spell a name letter by letter (14 letters) and insert it into the sentence ("My name is …").
F5	Teaching a new word	Add a new word or adapt to a new signer from 1–3 samples per word, without retraining a model.
F6	Speech and feedback	Show a banner with the word and its confidence, draw a motion trail and speak the result without blocking the video loop.
N1	Real time on CPU	At least 25 FPS in the live loop, text-to-speech dispatch latency below 50 ms.
N2	Explainability	Every decision can be traced to a distance to a stored template; no hidden training step.
N3	Reproducible evaluation	All reported numbers come from scripts in the repository with leave-one-session-out or unseen-signer protocols.

1.3. Scope and subjects

Table 2 – Scope of the current version

Item	Current state
Dynamic vocabulary	13 words in the frozen production bank: HELLO, BYE, ME, YOU, THANK_YOU, EAT, DRINK, WATER, SORRY, PLEASE, YES, BAD, HUNGRY. Five extra one-hand words (LIKE, NO, WANT, MY, NAME) recorded by one volunteer form the 18-word demonstration bank. NICE and MEET have sentence rules but no templates yet.
Nature of the new words	The five extra words are simplified one-hand conventions derived from ASL (several are two-handed in standard ASL); they are not standard ASL.
Fingerspelling	14 letters (A B C D E F G H I K L O S W) plus two hand-shape classes (1, 5), recorded by one signer.
Data	10 signer profiles; five people have enough data for evaluation (the owner, Mai, Phi, Dang, Vuong). The owner's bank holds 175 samples (7 recording sessions).
Hardware, interface	A webcam and a CPU-only Windows laptop; desktop window and a web version.
Constraints	Only one hand is tracked; the signer's face must be inside the frame (body-landmark features use the face box).

1.4. Structure of the report

Part 2 covers related solutions, tools, image-processing techniques and DTW. Part 3 presents the design and the replaced methods. Part 4 reports the experiments and, in Section 4.7, the evidence behind each replacement. Part 5 concludes.

PART 2. BACKGROUND KNOWLEDGE

2.1. Related solutions

Sign language recognition is usually attacked in one of five ways. Table 3 compares them against the constraints of this project (webcam, CPU, few samples, new signers).

Table 3 – Families of sign recognition solutions

Family	Principle	Strengths	Weaknesses for this project
Sensor gloves	Bend and motion sensors on the hand	Accurate finger state	Special hardware; not a camera-only product
Video deep networks (3D-CNN, transformers)	Learn directly from RGB clips	State of the art with large corpora, e.g. ASL Citizen [8]	Needs thousands of clips and a GPU; poor with a few samples per word
Image CNN on a hand crop	Classify a cropped, pre-processed hand image	Simple for static letters	Sensitive to light, background, blur and to the training domain; weak on dynamic signs
Landmark + sequence network (MLP, GRU)	Classify the landmark time series	Compact, accurate for known words	Needs training for every vocabulary change; weaker at refusing unknown words (Section 4.2)
Landmark + template matching (DTW)	Align the query with stored examples	1–3 samples per word, no training, explainable	Hand-tuned weights and thresholds; accuracy depends on who made the templates

The project adopts the last family for isolated words and uses small learned models only as auxiliary components (spotter, letter classifier, optional fallback). The experiments in Part 4 are the justification for this choice: an image CNN, trajectory-only features, a pure GRU and several tuning strategies were built or tested first and then replaced or rejected.

2.2. Tools and development environment

Table 4 – Software stack

Component	Version	Role
Python	3.12	Implementation language of every module
OpenCV (contrib)	>= 4.10	Camera capture, filtering, CLAHE, Canny, Sobel, drawing the interface [6]

Component	Version	Role
MediaPipe HandLandmarker	>= 0.10	21 hand landmarks per frame [1]; BlazeFace detector for the face box
NumPy / SciPy	>= 2.0 / >= 1.13	Vectorised feature construction, DTW, interpolation
PyTorch	>= 2.2	Spotter, letter classifier, GRU/MLP baselines, optional DL fallback [10]
scikit-learn	>= 1.4	Metrics (AUROC), data splitting
pyttsx3 + Windows SAPI	>= 2.90	Speech synthesis in a background thread
FastAPI / Uvicorn	>= 0.110	Web version of the interface
pytest	>= 8.0	485 passing automated tests (Section 4.8)
matplotlib / pandas	>= 3.8 / >= 2.2	Evaluation reports and charts

2.3. Hand landmarks

MediaPipe Hands [1] runs a palm detector followed by a landmark regressor and returns 21 three-dimensional keypoints per hand (wrist and four joints per finger), the handedness label and a set of metric "world" coordinates. Landmarks remove background, lighting and skin colour from the problem: two signers with different skin tone in different rooms produce comparable coordinates. This is the main reason landmarks replaced raw-image classification in the word loop (Section 4.7.1).

2.4. Digital image processing techniques used

Table 5 – DIP techniques and where they are used

Technique	Purpose	Used in
Bilateral filter [11]	Remove sensor noise while keeping the hand edges	`improved_dip_pipeline.py` (data collection, letter images)
Gray-World white balance [12]	Reduce colour cast from different lamps	Same pipeline
CLAHE [4]	Local contrast equalisation under uneven light	Same pipeline (channel 1 of the 128×128×3 tensor)
Canny edges [5] and Sobel gradients	Contour and gradient channels of the hand image	Channels 2 and 3 of the tensor
Image moments	Estimate the hand axis and rotate the crop upright	Smart hand crop
Scale normalisation by palm length	Make features independent of the distance to the camera	All trajectories and finger angles
Exponential moving average (EMA)	Smooth wrist jitter before velocity estimation	Segmenter (alpha about 0.65)

Technique	Purpose	Used in
Velocity thresholds on normalised motion	Detect start, pause and end of a sign	`KineticPauseSegmenter`
Linear resampling to K = 32	Compare gestures of different duration	Before DTW

2.5. Dynamic Time Warping

Two executions of the same sign never have the same speed. Dynamic Time Warping [2][3] finds the monotonic alignment between two sequences that minimises the accumulated local distance. For a query Q and a template T, both of length K:

$$D(i, j) = d(q_i, t_j) + \min\{ D(i-1, j), D(i, j-1), D(i-1, j-1) \}$$

The Sakoe–Chiba band [2] restricts the path to $|i - j| \leq r$. With K = 32 and r = 4 only about 32 × 9 = 288 cells are evaluated per comparison, which is why the matcher is fast enough for CPU real-time use. The local distance d is a weighted sum over feature groups (Section 3.5). A band of r = 4 also means the matcher cannot stretch one repetition into two, which is the structural reason a repetition-count feature was studied during the move to the 24-D layout (Section 4.7.2).

2.6. Evaluation protocols and metrics

- **LOSO (leave-one-session-out):** a recording session is held out and the other sessions form the reference bank. It measures generalisation within the bank owner's own signing.
- **Unseen signer:** all samples of one volunteer are queries and none of them is in the bank. It measures the realistic "stranger" case.
- **LOWO (leave-one-word-out):** a whole word is removed from the bank and its samples become unknown-word queries. It measures open-set rejection: the correct answer is not to accept.
- **Top-1** (forced choice), **correct-accepted** (ACCEPTED and correct), **wrong-accepted**, **AUROC** (how well the confidence separates known from unknown words, independent of any threshold) and **McNemar's exact test** [13] for paired comparisons.

PART 3. ANALYSIS AND SOLUTION DESIGN

3.1. System architecture

The system is a four-layer pipeline (Figure 1). Layer 1 turns each camera frame into normalised hand landmarks. Layer 2 decides when a sign starts and ends and blocks anti-patterns such as idle motion. Layer 3 builds a 24-D trajectory, resamples it to 32 frames and matches it against stored templates with DTW. Layer 4 collects words into sentences, composes natural language and gives audio-visual feedback.

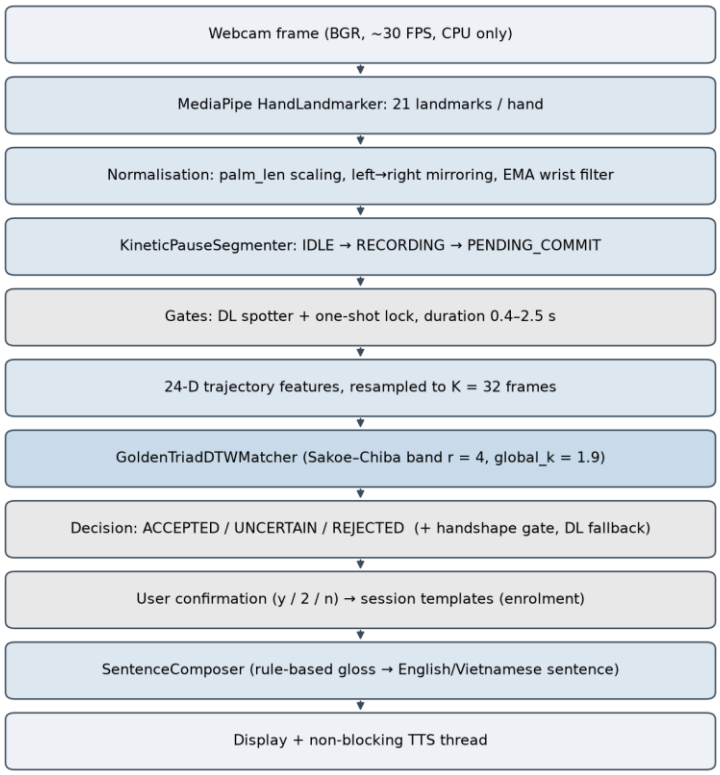


Figure 1 – Real-time pipeline from webcam frame to spoken sentence

The whole workflow fits in one short loop. Listing 1 is the per-frame routine of `pipeline.py`; every call is one of the eight steps of Figure 15 and is implemented by the module named in Table 24.

Listing 1 – Per-frame routine of pipeline.py (one call per processing step)

```
def step(self, frame, t):
    hand = self.perceive(frame, t)                        # 1  MediaPipe landmarks, palm_len,
angles
    if hand is None:
        return self.idle(t)                               #    no hand: close spans, finish
sentence
    stroke = self.segment(hand, t)                        # 2  velocity state machine -> stroke
or None
    self.spot(hand, t)                                    # 3  is the user signing? (1D-CNN,
gate)
    if stroke is None:
        return None
    records = self.featurize_records(stroke)              # 4  duration and transition filters
    traj = self.featurize(records) if records else None   #    24-D features, resampled to K =
32
    if traj is None:
        return None
    result = self.match(traj, hand)                       # 5  DTW against the template bank
    result = self.verify(result, records, hand)           # 6  hand-shape gate (uncertain
cases)
    return self.commit(result, stroke, t)                 # 7  one-shot lock, then sentence (8)
```

3.2. Design decisions at a glance: what was replaced and why

The current design is the result of several rounds of building, measuring and replacing. Table 6 lists each replacement together with the deciding evidence; the full numbers are in Section 4.7. Two principles governed every decision: **a change is accepted only if a paired, pre-declared measurement supports it**, and **rejecting an unknown input is worth more than guessing**.

Table 6 – Methods replaced or rejected, and the evidence

Component	Earlier method	Current method	Deciding evidence (details in Section 4.7)
Hand shape in the word loop	Two-stream image CNN on 128×128 CLAHE/Canny/Sobel tensors (5 letters)	Landmark-based classifier and handshape gate	CNN confused C with L and V with L even on its own training images (V only 63.0%); it never produced a usable label in 10,292 recorded frames and failed in live tests. Landmark model covers 14 letters, needs no image preprocessing.
Trajectory features	4-D (x, y, motion + CNN shape label), then 9-D/12-D	24-D landmark features, 5 groups active	Without finger angles accuracy fell 79.9% → 62.6%; trajectory only 33.8%; adding the CNN shape label dropped 79.9% → 54.7%; body landmarks separate signs that differ only in location.
Group weights	Random search over weights; ASL-LEX phonological weights	Fixed hand-set weights chosen through a paired scoreboard (`G1p90`)	ASL-LEX weights correlate with the data at +0.05 ($p = 0.63$); full ASL-LEX blending cut top-1 from 79.9% to 62.6%; random search over ~80 queries overfits.

Component	Earlier method	Current method	Deciding evidence (details in Section 4.7)
Acceptance threshold	Per-word threshold (scale_k 1.75 × each word's own width)	One global threshold (`global_k = 1.9`)	Unseen signer accepted 31/51 → 37/51 with no loss; unknown words wrongly accepted 3/175 → 0/175 (1.6% instead of 3.1% on random sub-banks); AUC 0.970 → 0.985.
Segmenter timing	Velocity thresholds counted in frames	Thresholds in seconds (`time_norm`)	At 60 fps correct segmentation 61.6% → 79.9% and missed strokes 34.6% → 6.9%; at 15 fps false swipes 9 → 0; unchanged at 30 fps.
Bank augmentation	7 synthetic variants per stroke	Original samples only	Unknown-word false accepts rose 3 → 5 of 175 and unseen signer gained 1 of 51; not used at runtime.
Main recogniser	GRU / MLP classifier (and a fixed-alpha DL/DTW blend)	DTW as the decision maker; DL only as a gated fallback	GRU equals DTW on forced-choice accuracy but accepts 12/175 unknown words (DTW 0/175); the fixed blend overwrote correct DTW answers and equalled the DL accuracy exactly.
Adapting to a new person	Pooling other signers / ASL Citizen into the bank; unsupervised self-learning	Per-person enrolment with 1–3 samples and user confirmation	Enrolment +26 to +30 points; pooling other people +4 or even −11; Citizen videos accepted correctly only 22.1%; self-learning gained 3 points.

3.3. Perception and normalisation (Layer 1)

Each frame goes through MediaPipe HandLandmarker, which returns 21 landmarks and the world coordinates. Three normalisations make signers comparable:

- **Palm-length scaling.** `palm_len` is the distance from the wrist to the base of the fingers. Every distance is divided by it, so a hand close to or far from the camera yields the same numbers.
- **Chirality.** A left hand is mirrored so that one sign produces one feature sequence regardless of the hand used. The live webcam is mirrored by default so that the geometry matches the training data of the spotter and the letter model.
- **Wrist smoothing.** The wrist coordinates pass through an exponential moving average (alpha about 0.65) to suppress landmark jitter before velocity is computed.

3.4. Segmentation and anti-false-trigger (Layer 2)

The normalised wrist speed drives a finite-state machine (`KineticPauseSegmenter`):

$$v_norm = \Delta w / palm_len$$

- `IDLE → RECORDING` when v_norm rises above the start threshold; frames are buffered.
- `RECORDING → PENDING_COMMIT` when the hand pauses. The machine waits 0.20 s: if movement resumes, the strokes are merged (multi-phase signs), otherwise the stroke is committed.
- A duration gate keeps only strokes between 0.4 s and 2.5 s. A fast leftward motion with a short buffer (at most 10 frames) is a **Swipe-Left**, which deletes the last word.
- A DL **spotter** (small binary network: is the user signing?) and a **one-shot lock** allow only one committed word per signing span. The spotter was trained on the 13 original words, so it can block new words; the demonstration bank therefore runs with the rule-based spotter (`--spotter rule`).

Time normalisation. Cameras rarely deliver exactly 30 fps. The velocity thresholds and the swipe detector are therefore expressed per second and scaled by the measured frame interval (`time_norm`), and exactly duplicated frames can be skipped. This change is evaluated in Section 4.5.

The two components that keep everyday movements out of the sentence are a hysteresis gate on the spotter probability and a one-shot lock that allows a single word per signing span. Listing 2 states their rules (constants are the values used in production).

Listing 2 – Spotter gate and one-shot lock (rules of spotter_gate.py)

```
# ProbGate: hysteresis on p = P(signing), updated on every frame
if gate is closed and p >= 0.50 for 0.10 s:  open the gate
if gate is open   and p <  0.35 for 0.20 s:  close the gate


# OneShotLock.allow_commit(t_stroke_start, word)
span = signing span that contains t_stroke_start
if span is None:                              # the spotter said 'not signing'
    allow only if max p in [t_start - 0.3 s, now] >= 0.25   -> reason 'soft'
    otherwise block                                         -> reason 'no_span'
if span already used:
    allow only if word != last_word and t_start - last_commit >= 1.2 s  -> 'next_word'
    otherwise block                                         -> reason 'dup_in_span'
mark span as used; allow                                    -> reason 'first'
```

3.5. Trajectory features (Layer 3, part 1)

For every frame of a committed stroke the system builds a 24-D vector (`dynamic_features.py`). The whole stroke is then resampled by linear interpolation to K = 32 frames, so every sign has the same length regardless of speed.

Table 7 – Feature groups of the 24-D vector and weights used in production (G1p90.json)

Group	Columns	Weight	Meaning
Position (stroke-anchored)	0–1	0.6	Wrist x, y relative to the first frame, divided by palm_len
Velocity	2–3	0.4	Frame-to-frame wrist displacement
Finger flexion angles	4–8	1.0	Five finger bend angles – the most informative group
Palm normal	9–11	0.8	Orientation of the palm (cosine distance)
Angular velocity	12–16	(with angles)	Change of the five finger angles
Virtual depth z_norm	17	0.0	Implemented and measured, switched off
Body landmark	18–19	0.8	Wrist position relative to the face box (forehead, chin, chest, mouth)
Finger spread	20–23	0.0	Implemented and measured, switched off

Each group is divided by the square root of its dimension so that groups with more columns do not dominate silently. The two groups that were implemented but set to zero (depth, finger spread) are an example of the rule "measure every group and keep only those that raise the margin".

3.6. Matching and the three-level decision (Layer 3, part 2)

`GoldenTriadDTWMatcher` computes the weighted DTW distance between the query and every template of the active bank (`webcam_templates.json`, `demo_templates.json` or a per-person file in `profiles/`). The best word is the one with the smallest distance d_1. The decision uses a global threshold $\theta = k \times w$, where w is a shared intra-class width and k = 1.9 (`global_k`):

Table 8 – Decision rules

Outcome	Condition	System reaction
ACCEPTED	$d_1 \leq \theta$ and the margin to the second best word is at least 12%	Word is committed, displayed and spoken
UNCERTAIN	$\theta < d_1 \leq 1.4\ \theta$, or the top-2 candidates are closer than 12% (4% when d_1 is already close to θ)	The user is asked to confirm (y / 2 / n)
REJECTED	$d_1 > 1.4\ \theta$	Nothing is committed

Listing 3 gives the three-level decision of the matcher in pseudocode. The width is the median intra-class DTW distance of the bank (global width, k = 1.9); the margin guard asks for a clear gap between the first and second word unless the match is very close.

Listing 3 – Three-level decision of the DTW matcher (pseudocode of GoldenTriadDTWMatcher.match)

```
for each word w in the bank:
    d[w] = min over templates of DTW_band(query, template)      # Sakoe-Chiba band, 24-D
features
rank words by d;  top1, top2 = best two words
accept_limit = k * width          # k = 1.9, width = global intra-class width
reject_limit = uncertainty_k * width   # uncertainty_k = 2.5
if d[top1] > reject_limit:                                      return REJECTED
margin = (d[top2] - d[top1]) / d[top1]      # relative contrast margin
required = 0.04 if d[top1] <= 0.70 * accept_limit else 0.12
if margin < required:                                           return UNCERTAIN   # two words too
close
if d[top1] <= accept_limit:                                     return ACCEPTED
return UNCERTAIN                                                # between the two limits
```

3.7. Auxiliary layers

- **Handshape gate** (`handshape_gate.py`): a table, built from the data, lists which hand shapes occur in each word; a candidate whose hand shape is not allowed is removed. It uses the landmark letter classifier, not an image CNN.
- **DL fallback gate** (`dl_fusion.py`, optional `--dl-model`): a small 1D-CNN + Bi-GRU + attention network (`LightweightGestureNet`) is consulted only when DTW is UNCERTAIN or REJECTED, and its answer is accepted only if its softmax confidence is at least 0.38. A DTW ACCEPTED result is never overridden.
- **Spotter and one-shot lock** (`spotter_dl.py`, `spotter_gate.py`): described in Section 3.4.

3.8. Enrolment and confirmation

A new word or a new signer is taught with `dip.py hieu-chinh`: the user signs one to three samples per word and they are saved to `profiles/<name>.json`. The production bank of the owner is never written. At run time a profile uses the owner's bank plus the person's own templates. During live use, an UNCERTAIN or REJECTED result with a candidate triggers a question; a confirmed sample is added to the session bank, so the system improves within the same session without training.

3.9. Fingerspelling

Key `s` switches to spelling mode (`fingerspell.py`). A landmark MLP (`letter_landmark_model.pt`) classifies the static hand shape of each frame from normalised 3-D landmarks (`hand_pose_features.py`). A letter is committed when the same letter is held stable for about 0.7 s with confidence ≥ 0.6; a repeated letter needs the hand to drop or the shape to change for about 0.35 s. The result is a token `SPELL:<NAME>` that the sentence composer turns into "My name is …".

3.10. Sentence composition and output (Layer 4)

Committed words accumulate in a sentence. The sentence is closed when the hand is lowered for more than 1.8 s or when eight words are collected. `sentence_composer.py` applies offline rules (no language model) that add pronoun forms, negation and objects and never delete a content word; if no rule applies the words are joined as they are. Examples are shown in Table 9.

Table 9 – Examples of gloss-to-sentence rules

Gloss sequence	English output	Remark
ME LIKE YOU	I like you.	Pronoun case
ME WANT WATER	I want water.	Verb + object
ME NO LIKE YOU	I don't like you.	Negation with NO
MY NAME SPELL:ANNA	My name is Anna.	Name from fingerspelling
NICE MEET YOU	Nice to meet you.	Fixed phrase

Feedback runs without blocking the video loop: a large banner shows the English word and its confidence for 2.0 s, a fading neon polyline draws the hand trajectory, and `pyttsx3` speaks the word from a background daemon thread guarded by a non-blocking lock (dispatch latency below 1 ms, loop latency below 50 ms).

PART 4. EXPERIMENTS, EVALUATION AND DISCUSSION

4.1. Experimental setup

All experiments were run on Windows laptops without a discrete GPU, with Python 3.12 and the library versions in Table 4. Every number in this part comes from a script stored in the repository (`eval_baselines.py`, `eval_learning_curve.py`, `eval_hybrid.py`, `eval_enroll.py`, `eval_confirm.py`, `eval_open_set.py`, `tools/eval_segmenter_fps.py`, `tools/eval_handshape_gate.py`, `train_letter_landmarks.py`) and its raw output is kept under `results/`.

Table 10 – Data used in the experiments

Data set	Size	Use
Owner bank (`webcam_templates.json`)	13 words, 175 samples, 7 sessions	Reference bank; LOSO queries; frozen
Volunteer Mai	51–100 samples	Main unseen-signer test (left-handed signer)
Volunteer Phi	27–48 samples, 8–13 words	Unseen signer; also records the 5 new words
Volunteers Dang, Vuong	42 and 52 samples	Unseen signers for confirmation and hybrid tests
Negative strokes	41 strokes	Everyday movements that must not become words
ASL Citizen subset [8]	226 trimmed videos	External, out-of-domain measurement only
Letter landmarks	434 real test images, 16 classes	Letter classifier evaluation (one signer)
Raw strokes	10,292 frames, 14 words	Check whether the image CNN ever produced a label

Protocol. Closed-set results use LOSO (7 folds). Unseen-signer results use all samples of a volunteer as queries against the owner's bank. Open-set rejection uses LOWO (175 queries) and the 41 negative strokes. Learned baselines use the same bank, the same folds and the same 32×24 trajectories as DTW, on CPU only, and three random seeds.

4.2. Word recognition: DTW against learned baselines

The first experiment asked whether a trained classifier would be a better recogniser than DTW. Table 11 reports forced-choice accuracy and the ability to separate known from unknown words (AUROC); Table 12 fixes every method at the operating point where it accepts the same number of correct LOSO queries (152) and counts the errors.

Table 11 – Closed-set accuracy and unknown-word discrimination

Method	LOSO top-1	Mai top-1	AUROC all	AUROC unknown word	AUROC negative
DTW `G1p90` (current)	100%	92.2%	0.990	0.990	0.994
DTW `C11_base` (old threshold)	100%	92.2%	0.990	0.989	0.991
1-NN Euclidean	95.4%	68.6%	0.804	0.812	0.769
Small MLP (3 seeds)	97.5 ± 0.7%	72.5%	0.918	0.903	0.980
Small Bi-GRU (3 seeds)	100%	94.1 ± 5.5%	0.974	0.969	0.994

Table 12 – Operating point at equal LOSO acceptance (152 correct)

Method	LOSO wrong	Mai accepted (of 51)	Unknown words accepted (of 175)	Negatives accepted (of 41)
DTW `G1p90`	0	37	0	0
1-NN	8	34	107	35
MLP	0	31.3	28.7	1.3
GRU	0	29.0	12.0	0

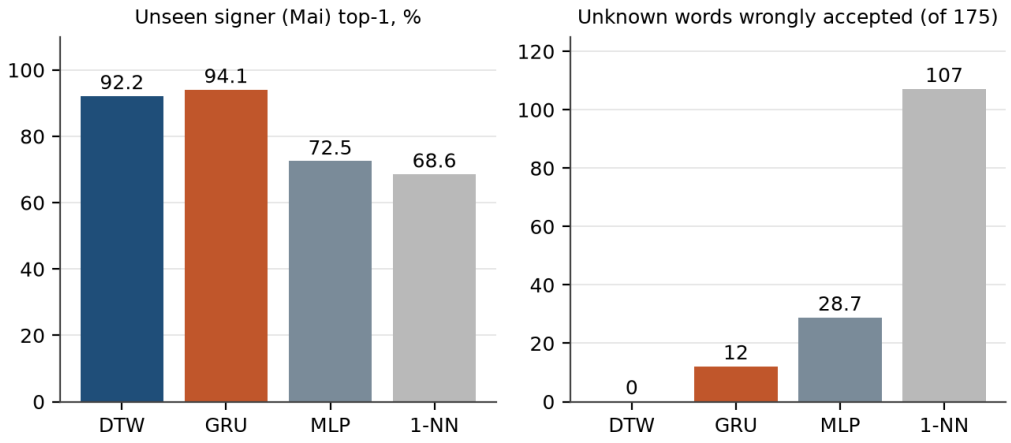


Figure 2 – Unseen-signer accuracy and wrongly accepted unknown words for four recognisers

Findings. (1) For forced choice the GRU is equal to DTW (the difference on Mai lies inside the ±5.5 noise). DTW is **not** more accurate than deep learning. (2) For refusing unknown words DTW is clearly better: 0 of 175 unknown words accepted against 12 for the GRU and about 29 for the MLP, and 37 against 29 correct acceptances for the unseen

signer. (3) Euclidean 1-NN is far worse, which confirms that temporal alignment is needed.

Learning curve. Figure 3 keeps only n samples per word on the reference side. The hypothesis "DTW beats deep learning when samples are few" is **not supported**: the GRU is equal or better at every n, even n = 1 (Mai 97.4% against 89.5%). DTW only beats the MLP and 1-NN. The remaining advantages of DTW are rejection of unknown words, no training and explainability, not raw accuracy.

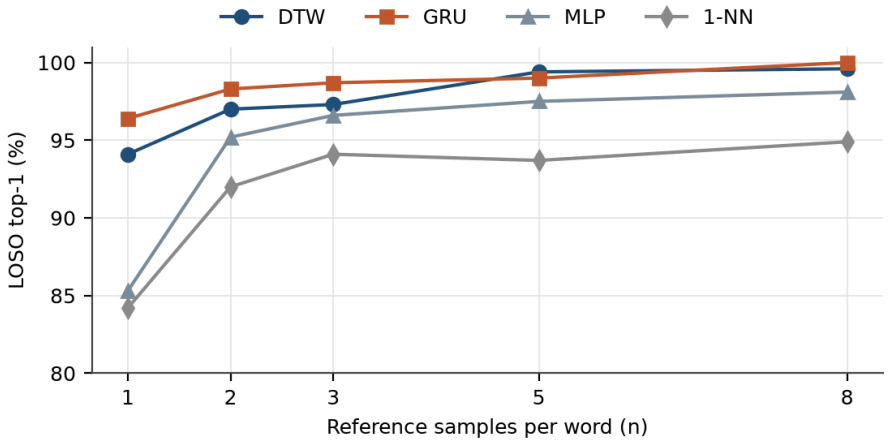


Figure 3 – LOSO top-1 against the number of reference samples per word

Word confusion. Using the trajectory only, the 13-word LOSO confusion matrix (189 queries) has a total accuracy of 99.5%. The only confusion in the whole vocabulary is WATER → DRINK (1 of 12). YES, which was thought to need an extra hand-shape letter, is already 100% correct from its movement.

4.3. Adapting to a new signer

A bank recorded by one person gives only 17–66% correct acceptance for a stranger (Dang 33.3%, Vuong 17.3%, Phi 58.5%, Mai 65–66%). Three ways of closing this gap were measured.

4.3.1. Enrolling the person's own samples

Table 13 – Correct and accepted (%) when k own samples per word are added (5 repetitions)

Person	k	Owner bank only	Owner + own	Own only	Owner + others	Others + own
Mai (~90 queries)	1	65.1	91.6	90.2	69.3	91.6
Mai	2	66.5	94.8	96.0	70.5	95.0
Mai	3	67.1	95.1	94.6	70.9	95.1
Phi (~27 queries)	1	58.5	84.4	88.1	47.4	79.3
Phi	2	62.1	93.7	95.8	50.5	91.6
Phi	3	70.0	100	100	56.7	98.3

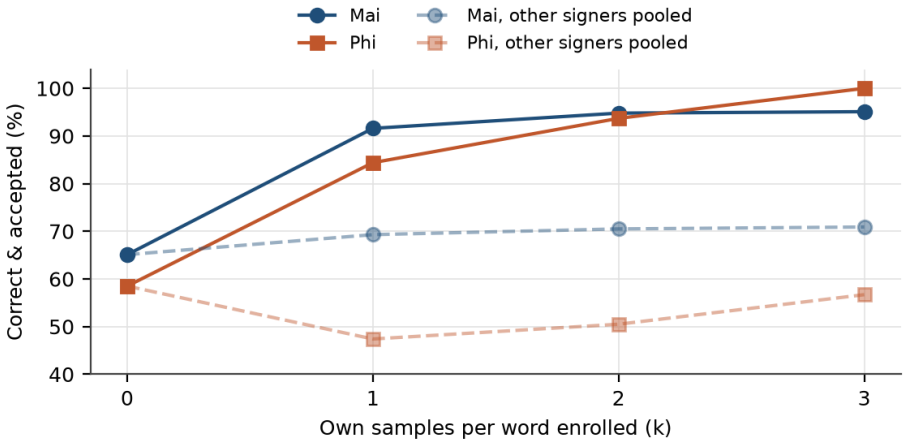


Figure 4 – Effect of enrolling a stranger's own samples compared with pooling other signers

One to three own samples per word (3–5 minutes) raise correct acceptance by 26–30 points with almost no wrong acceptances. Pooling **other** people's samples barely helps (Mai +4 points) and can hurt (Phi 58.5% → 47.4%). Keeping the owner's bank next to the own samples is safer than own samples alone because it acts as a reference zone: own-only banks accepted 0.8–5.4 of 41 negative strokes.

4.3.2. Confirmation questions during use

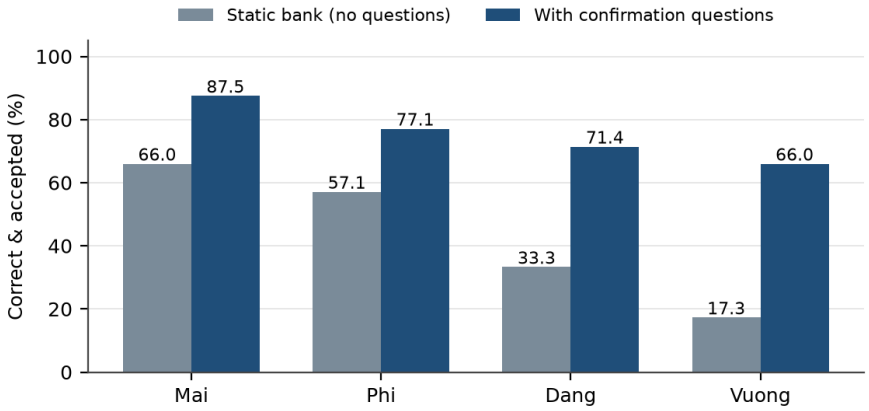


Figure 5 – Correct-accepted rate with and without confirmation questions in one session

When the system asks for confirmation on UNCERTAIN and REJECTED results (simulated, with a truthful user), only 8–16 questions per 35–100 signs are needed, and accuracy rises from 66.0% to 87.5% (Mai), 57.1% to 77.1% (Phi), 33.3% to 71.4% (Dang) and 17.3% to 66.0% (Vuong). Within the session the last quarter reaches 83–98%, and negative strokes wrongly accepted stay at 0–1 of 41. Asking only on UNCERTAIN is clearly worse (Dang 55.6%, Vuong 30.8%), so REJECTED results must also be asked about.

4.3.3. Pooling more real signers

When real volunteers are added as references for a held-out signer, the top-1 accuracy rises for all five people: DTW rises quickly from one to three reference people and then saturates (Phi 83.3% → 97.9%, Dang 75.0% → 95.2%). About three reference people already capture most of the benefit. This differs from simply pooling at the sample level in Table 13, because here the person is unseen and the extra people supply style diversity rather than own samples.

4.3.4. External data and unsupervised self-learning

On 226 ASL Citizen videos (out of domain) the owner's bank accepted only 22.1% correctly (top-1 61.6%) but made only 1.2% wrong acceptances and accepted 2 of 53 unknown words; it refuses instead of guessing. Merging Citizen data into the bank increased correct acceptance, but wrong acceptances also rose from 0% to 6.5–9% because the threshold was not recalibrated. Unsupervised self-learning (adding confidently accepted samples under the predicted label) lifted Mai from 66.0% to only 69.3% and left Phi unchanged, since it cannot learn exactly the hard samples that are rejected; without a gate it reached 73% but made 10% wrong acceptances.

4.4. Gated deep-learning fallback

A `LightweightGestureNet` was trained on the pooled samples of the other signers and consulted only when DTW was UNCERTAIN or REJECTED. The confidence threshold 0.38 was chosen by nested cross-validation, i.e. for each held-out person using only the other three, so that no data of the tested person influenced the threshold; the mean over the four people (96.6%) equals the result with the fixed global threshold.

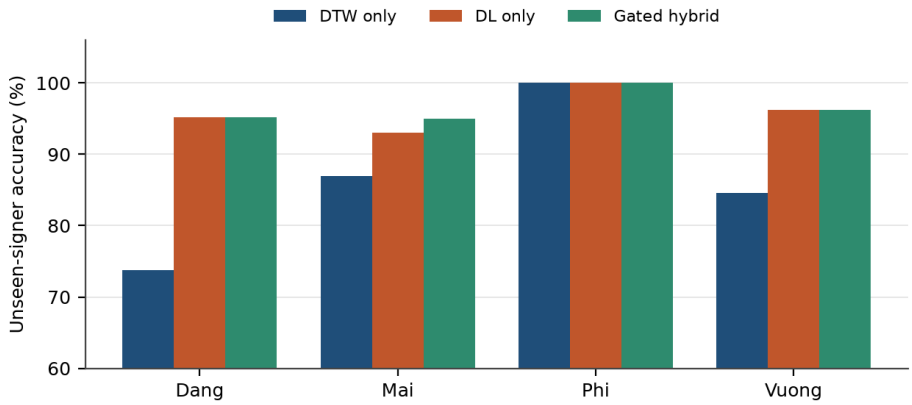


Figure 6 – Unseen-signer accuracy of DTW, deep learning and the gated hybrid

The hybrid exceeds both single systems for Mai (95.0%; e.g. DRINK: DTW 17/18, DL 16/18, hybrid 18/18). Over three training seeds the DL accuracy varies by only 0–2.9 points. The benefit is largest before enrolment (+8 to +21 points at k = 0) and shrinks as enrolment grows; for Mai with $k \geq 1$ the gate even lost 0.4–2.3 points. Therefore the fallback is optional and is most useful for a first-time user. Only 2 of 242 queries (0.8%) were confident-but-wrong DTW answers, which the gate by design never overrides.

4.5. Robustness of segmentation to camera frame rate

The segmenter was tested on 159 real strokes replayed at seven frame-rate variants without a camera (`tools/eval_segmenter_fps.py`). Table 14 compares the frame-count thresholds with `time_norm`.

Table 14 – Segmentation with frame-count thresholds against time_norm

Variant	Before (frame-count)	After (`time_norm`)
60 fps	61.6% correct, 34.6% missed	79.9% correct, 6.9% missed
45 fps	76.7% correct, 17.0% missed	86.8% correct, 5.7% missed
15 fps	91.8% correct, 9 false swipes	97.5% correct, 0 false swipes
Duplicated frames	9 false swipes	0 false swipes
30 fps (reference)	90.6%	90.6% (identical)

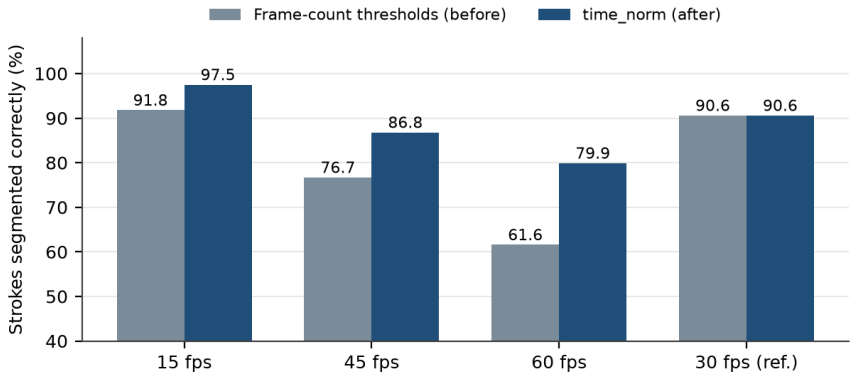


Figure 7 – Share of strokes segmented correctly at different frame rates

A simulated Swipe-Left (9 palm lengths per second, 0.3 s) was missed completely at 60 fps by the old thresholds and detected by the new ones at every frame rate from 8 to 60 fps. The price is a slightly higher rate of split strokes at 60 fps (3.8% → 13.2%), accepted because missed strokes fell much more. The setting is on by default since 2026-09-29.

4.6. Hand shape and fingerspelling

The landmark letter classifier (16 classes) reached **84.8%** on 434 real test images. Thirteen classes were recognised at or near 100%; the weak classes are `b` (37.7%, mostly confused with `5`), `1` (28.6%, confused with `d`) and `a` (0%, confused with `o`). Training without augmentation gave 80.4% on the same images and only 62.2% on the synthetic stress set, against 84.6% with augmentation, so augmentation is kept for this model. Letter data comes from a single signer, so the figure is not a stranger result.

As a gate for words, the handshape table was evaluated on 242 queries from four people: correct acceptances rose from 117 to 123 and wrong acceptances from 2 to 3. This is a small, non-decisive gain, which is why the gate is a filter and not a recogniser.

4.7. Why the current methods replaced the earlier ones

This section collects the evidence for every replacement summarised in Table 6. Each item states the earlier method, the test, the result and the reason for the change. Where a result is limited (few people, hand-tuned parameters) this is said openly.

4.7.1. Image CNN → landmark-based hand shape

Earlier method. `TwoStreamHandCNN` took a 128×128×3 tensor (CLAHE, adaptive Canny, Sobel) of the smart hand crop. It was trained on 451 still images of five letters (A 125, B 155, C 64, L 53, V 54) and reached 92.3% validation accuracy.

Table 15 – Per-letter accuracy of the image CNN on all stored images (including training images)

Letter	Correct	Accuracy	Confused with
A	135 / 135	100.0%	–
B	154 / 155	99.4%	1 → C
C	108 / 115	93.9%	6 → L, 1 → B
L	52 / 53	98.1%	1 → V
V	34 / 54	63.0%	20 → L

Test and result. In the first live test with the CNN switched on, the hand shape C (the shape of DRINK) was displayed mostly as L or B. The cause was investigated in two layers: (i) integration – an outdated checkpoint path (v2 instead of v3, +1.5 points) and no filtering of motion-blurred frames; a stable-frame filter (speed ≤ 0.04 and confidence ≥ 0.55) was implemented but did **not** fix the live result; (ii) the model itself – Table 15 shows that even on its own training images C is confused with L and V is only 63% correct, a weakness hidden by the 92.3% average. A scan of 10,292 recorded frames of 14 words found **no frame with a real hand-shape label**: the CNN had never contributed to any stored recording. Class weighting was added to training, but the domain gap (still studio images against moving webcam hands) and the thin classes remained.

Reason for the replacement. Landmarks remove lighting, background and skin colour, cover 14 letters instead of 5 with a small MLP, and need no image preprocessing in the live loop (`enable_static_shape_cnn = False`). The landmark model is not perfect (Section 4.6: `a`, `b`, `1` are weak), so the argument is robustness and coverage, not a higher number on a common test set; the two models were measured on different data. The image pipeline is retained only for data collection and letter-image experiments.

4.7.2. Trajectory + shape label → 24-D landmark features

Earlier method. The first engine (`HybridDTWEngine`) used a 4-D vector (position, speed) plus a per-frame CNN shape label; later versions grew to 9-D and 12-D.

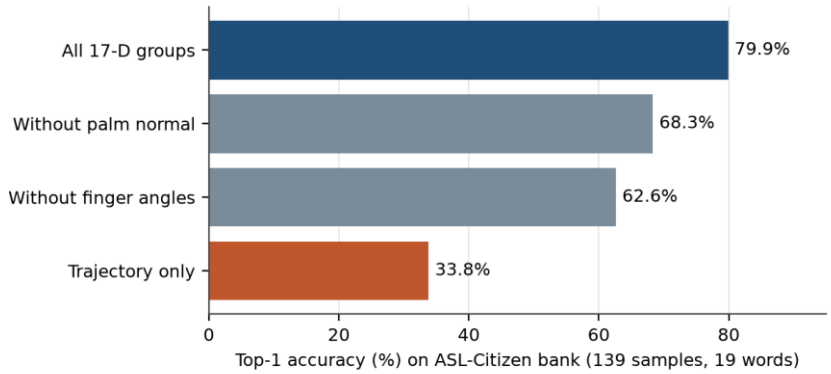


Figure 8 – Ablation of feature groups on the ASL Citizen bank (139 samples, 19 words)

Test and result. An ablation (Figure 8) shows that hand configuration is the most valuable channel: removing finger angles drops top-1 from 79.9% to 62.6%, removing the palm normal to 68.3%, and keeping only the trajectory leaves 33.8%. Adding the CNN shape label as one more channel made the result worse (79.9% → 54.7%) because a noisy group dilutes the weight of useful groups. Two structural blind spots were also measured: without a body reference the signs FATHER (forehead) and MOTHER (chin) differ only in location and cannot be separated mathematically (MOTHER 2/4, FATHER 8/11 in leave-one-out), and YES and BYE collide because a Sakoe–Chiba band cannot count repetitions. Adding depth, body landmark and finger spread raised the mean margin from 0.10 to 0.23 on that bank.

Reason for the replacement. The 24-D layout replaces shape labels with continuous finger angles and palm normal that do not depend on a classifier, and adds the body landmark. In production only the groups that survived the paired scoreboard keep a non-zero weight (position, velocity, angles, palm normal, body landmark); depth and finger spread are implemented but weighted zero.

4.7.3. Searched or phonological weights → fixed, scoreboard-chosen weights

Table 16 – Test of ASL-LEX phonological weights on 10 words (68 Citizen samples)

Measurement	Result	Meaning
Rank correlation of ASL-LEX group weights with groups that really separate the words	+0.05 (p = 0.63)	No better than chance
ASL-LEX phonological similarity against DTW distance for 36 word pairs	+0.05 (p = 0.61)	Does not predict confusable pairs
Emphasising the groups ASL-LEX says differ per pair	Δ log(inter/intra) −0.020 against −0.022 random	Not better than random labels
Replacing all weights by ASL-LEX (`asllex_blend_alpha = 1`)	Top-1 62.6% against 79.9%	Harmful

Reason. With about 80 queries and 8 weights, a 300-point random search can only overfit, and the knowledge-based alternative was not better than chance on this data. The weights were therefore fixed to a few named configurations compared pairwise on one scoreboard with the acceptance rule declared before the run. Caveat: the final weights and k were still chosen by the authors on this bank, so the DTW figures are somewhat optimistic compared with the baselines, which only had default hyper-parameters.

4.7.4. Per-word thresholds → one global threshold

Table 17 – Threshold strategies on the full bank (LOSO 175, Mai 51, LOWO 175, negatives 41)

Configuration	LOSO accepted	Mai accepted	Unknown accepted	Negatives accepted	Wrong (heavy)
`scale_k 1.75` (per-word, old)	155/175 (88.6%)	31/51 (61%)	3/175	0/41	0
`global_k 1.9` (current)	152/175 (86.9%)	37/51 (73%)	0/175	0/41	0
`global_k 2.0`	155/175 (88.6%)	38/51 (75%)	0/175	0/41	0

Result. The per-word width is a noisy estimate (10–17 samples of one person), so a very regular word such as BYE received a threshold that was too tight for other people, while other words still accepted unknown words. The AUC for separating the right word from unknown words rose from 0.970 to 0.985 (unseen signer from 0.898 to 0.963) with a shared scale. For the unseen signer 7 more samples were accepted and none was lost (McNemar p = 0.016). A sensitivity test on six random 70% sub-banks (732 queries each) showed that the "0/175" at full bank is partly luck: unknown-word false acceptance is 3.1% for the old rule, 1.6% for k = 1.9 and 2.7% for k = 2.0. At full bank a cliff exists at k = 2.05 (4/175). k = 1.9 was chosen as the safer value. The uncertainty factor must be at least k + 0.6 or the UNCERTAIN zone becomes too narrow.

Reason for the replacement. A shared scale halves unknown-word acceptance and raises acceptance for strangers by 12 points at a cost of 1.7 points of acceptance inside the owner's data (1.3 on the sub-banks). Limits: one volunteer, thresholds tuned on the same bank (mitigated by the sub-bank test), BYE of the volunteer is still 0/8 because that volunteer's signing style differs.

4.7.5. Frame-count segmenter → time_norm

Reason. Table 14 and Figure 7 show a clear improvement at every frame rate other than 30 and no regression at 30 fps, and one real bug (a missed Swipe at 60 fps) was fixed. The 23 robustness tests continue to pass. The change has been checked on replayed recordings, not yet on a live webcam at other frame rates.

4.7.6. Augmented templates → original samples only

Table 18 – Effect of adding synthetic variants (7 per stroke) to the bank

Reference	LOSO accepted	Mai top-1 / accepted	Unknown accepted	Latency
Original samples only	155/175 (88.6%)	47/51 · 31/51	3/175	4 ms
+ variants, strength 0.5	158/175 (90.3%)	47/51 · 31/51	5/175	27 ms
+ variants, strength 1.0	161/175 (92.0%)	46/51 · 32/51	5/175	26 ms

Reason. Time stretch, scale, ±10° rotation, start offset, frame drop and noise raise acceptance for the owner (+6 queries, none lost, $p = 0.031$), but the unseen signer gains only one sample, top-1 drops by one and unknown-word false acceptance rises from 3 to 5. The pre-declared rule "if wrong acceptance increases, drop it" applies. The explanation is simple: the bank is already saturated for the owner (top-1 100%), and the difference between signers is a difference in style that small perturbations cannot simulate. Augmentation also made the matcher about six times slower (4 → 26 ms), so it remains only a measurement tool (`augment_templates.py`). A by-product was a corrected width calculation that now ignores synthetic variants.

4.7.7. Deep learning as the main recogniser and the fixed-alpha blend → DTW with a gated fallback

Reason for keeping DTW as the decision maker. Section 4.2 shows equal closed-set accuracy but much better refusal of unknown words, no training when the vocabulary changes (the 18-word bank was created by recording samples, not by training) and decisions that can be explained by a distance. **Reason for gating the DL.** The first hybrid mixed probabilities with a fixed weight (0.65 DL + 0.35 DTW) and ignored the DTW status; for BYE, EAT and DRINK the hybrid accuracy equalled the DL accuracy exactly, i.e. it overwrote correct DTW answers every time. Replacing the blend by a status-based gate (never override ACCEPTED; ask the DL only otherwise; threshold from nested cross-validation) gave the results in Section 4.4. A simpler rescue variant that accepts UNCERTAIN results when the GRU agrees with DTW's word (GRU ≥ 0.8)

lifted LOSO acceptance from 152 to 169.7 with 0 wrong LOSO acceptances and 2 of 175 unknown words accepted, but it was not adopted because its threshold was tried on the same data and it adds a torch dependency; it is a documented future option.

4.7.8. Pooling other signers or external data → per-person enrolment

Reason. Table 13 and Section 4.3 show that own samples are worth +26 to +30 points, whereas other people's samples are worth +4 or −11 and external Citizen videos were accepted correctly only 22.1% of the time because of domain shift and unreliable trimming. The benefit of external data was inversely proportional to the amount of local data: it helped when the training pool was thin (the owner, 135 background samples: +24.6 points over a duplicate-data control) and not when it was thick (Mai and Phi, 220+ samples: −0.5 and 0.0). As the bank grows, the value of Citizen data decreases further, so manual trimming of more Citizen videos is not worth the effort. Enrolment with confirmation costs a few minutes and about 12–16 questions per person.

4.8. Automated software testing

`py -m pytest tests -q` ran in a single 228-second run on 2026-10-14: **485 passed**, 1 failed, 4 skipped and 7 expected failures. The single failure is `test_real_sapi_reads_every_queued_sentence`, which uses the real Windows SAPI engine and depends on timing and machine load (it passes when run alone); it is unrelated to code changes. A cross-drive path error in `rebuild_templates_from_raw.py` found by an earlier run was fixed. The suites cover the engine contracts (legacy 4-D and production engine), numeric parity of the fast DTW with the reference DTW, the segmenter state machine and fast sign chains, the 24-D features, the handshape gate, letter and spotter modules, 97 sentence-composer cases (22 new for LIKE/WANT/NO/MY NAME/NICE MEET) and 12 fingerspelling cases.

4.9. User interface and how to run the software

The desktop application is started through one entry point, `dip.py`. The camera window shows the video, the hand skeleton in debug mode, the motion trail and the banner; a right-hand side panel shows candidates and distances; a bottom panel shows the current sentence.

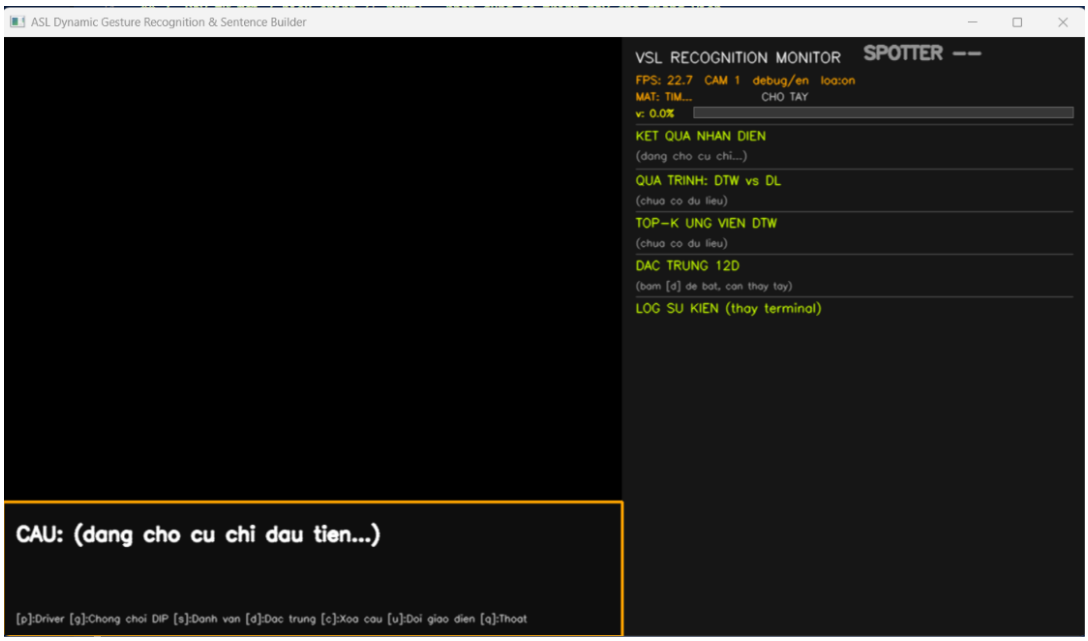


Figure 9 – Main window of the application in its idle state: camera view (left), recognition monitor (right) and sentence panel with key legend (bottom)

Figure 9 was captured while the system was waiting for the first gesture, so the camera area is dark and the result areas read "waiting". The window has three regions. **Camera view (left):** the video with the hand skeleton, motion trail and result banner. **Recognition monitor (right):** a status line (frame rate, camera index, UI mode, voice on/off, spotter state, hand-motion bar) followed by the recognition result, the DTW-versus-DL pipeline, the top-k DTW candidates, the 12-D feature view (toggled with `d`) and an event log that replaces the terminal output. **Sentence panel (bottom):** the sentence being built and the list of keys. The labels in the interface are written in Vietnamese without diacritics.

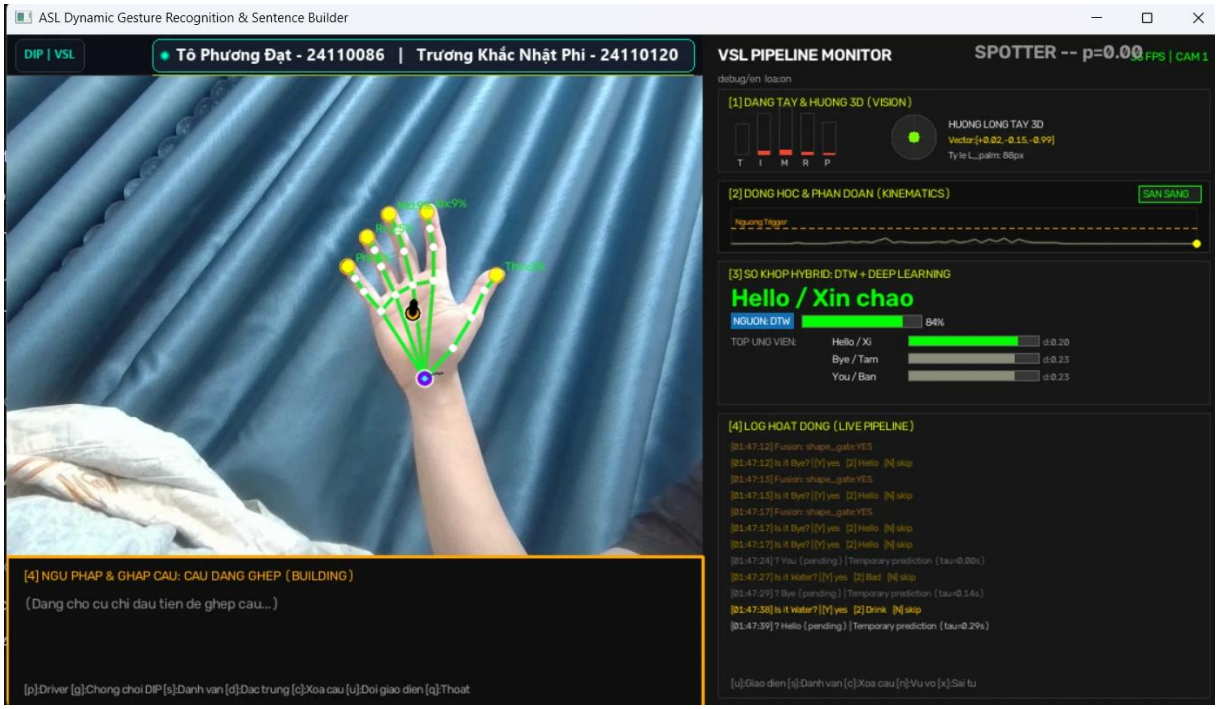


Figure 10 – Main window during live recognition: hand skeleton with per-finger labels (left), recognition monitor with the candidate list, the activity log and the spotter indicator (right), and the sentence panel (bottom)

Figure 10 was captured on the live camera with an open hand held still. **Panel [1], hand shape and 3D orientation:** five bars show the flexion of each finger, the circle shows the palm-normal vector (+0.02, −0.15, −0.99), i.e. the palm faces the camera, and the palm scale is 88 px. **Panel [2], kinematics:** the normalised-velocity trace stays under the trigger threshold (dashed line) and the state reads READY. **Panel [3], matching:** the temporary prediction is Hello (84%, source DTW) with the ranked candidates and their DTW distances (Hello 0.20, Bye 0.23, You 0.23). **Panel [4], activity log:** the confirmation questions ("Is it Bye? [Y] yes [2] Hello [N] skip") and the hand-shape-gate notes (shape_gate). The indicator in the top-right corner reads "SPOTTER -- p=0.00": the hand is not moving, so the spotter correctly reports that nobody is signing. The sentence panel at the bottom is empty because no word has been committed yet.



Figure 11 – The DIP 5-step pipeline inspector (key d): the image-processing chain applied to the hand crop, from the filtered crop to the Sobel gradient, and the three-channel tensor it produces

Figure 11 is the inspector window that opens with the key `d` and shows, for the current frame, the digital image processing chain applied to the hand crop (the labels are in Vietnamese; the techniques are those of Section 2.4 and Table 5). **[1] Bilateral filter** (d = 5, σ = 45) removes sensor noise while keeping the edges of the fingers. **[2] Skin segmentation** keeps the pixels whose CIELab (a*, b*) and YCrCb (Cr, Cb) values fall in the skin range and erases the background. **[3] Image moments:** the orientation of the main axis, $\theta = \frac{1}{2}\cdot\text{atan2}(2\mu11, \mu20 - \mu02)$, is used to rotate the hand upright (0°); the wrist is trimmed and the hand is centred on a black canvas, so the result does not depend on lighting or hand rotation. **[4] Canny edges** use thresholds derived from the median of the grey levels (automatic, one pixel wide). **[5] Sobel gradient** shows the gradient magnitude $G = \sqrt{(Gx^2 + Gy^2)}$ with a 3×3 kernel, which keeps the finger structure when the light is poor. The three results are stacked into the tensor shown in the bottom bar: channel R = CLAHE contrast, channel G = Canny edges, channel B = Sobel gradient (the small image at the right end). This tensor is the input of the two-stream hand-shape CNN described in Section 4.7.1; in the word-recognition loop the hand-shape decision now relies on landmarks instead, so the window is a diagnostic view of the image-processing chain and not a step of the default word decision.

```
py dip.py nhan-dien -- --cam 0 --spotter rule --templates demo_templates.json
py dip.py hieu-chinh --ho-so <name> --tu HELLO ME --so-mau 3
py dip.py web
```

Table 19 – Main controls

Control	Action
Sign a word	Sign, then pause; the banner shows the word and confidence and the voice reads it
Swipe left	Fast leftward motion deletes the last word

Control	Action
Lower the hand > 1.8 s	Closes the sentence and shows the composed English/Vietnamese sentence
y / 2 / n	Confirm the first candidate, choose the second, or reject when the result is UNCERTAIN
s	Toggle fingerspelling mode (hold each letter about 0.7 s)
d	Show or hide the feature view of the current stroke
c / z	Clear the sentence / undo the last confirmed sample
u	Switch between the demo and debug interface
n / x	Mark the last gesture as stray motion (saves a negative sample and removes a word committed in the last 3 s) / mark the last committed word as wrong (removes it and logs it)
f / t / l	Flip the camera / toggle the speaker / switch the language (EN/VI)
g / p / j	Anti-glare enhancement / camera driver settings / fast-signing mode
q	Quit

Suggested demonstration order: sign two to five single words, then a sentence such as ME WANT WATER, then MY NAME with fingerspelling, and finally teach one word to a new person with three samples to show that no training is needed. Practical notes: keep the face and the hand both in view, use `--spotter rule` for the 18-word bank, and wait for the SAVED banner before signing again after teaching a sample.

4.10. Discussion and limitations

- **Few signers, few words.** The owner's bank is one person, only four to five people were measured, and per-person sample counts are small (27–100); confidence intervals are wide (GRU on Mai varies by ±5.5 points).
- **Hand-tuned DTW.** Group weights and k were chosen by the authors on the evaluation bank, whereas learned baselines had default hyper-parameters. The conclusions about rejection are robust to this (sub-bank test), the conclusions about accuracy are not.
- **Optimistic enrolment figures.** Enrolment and query samples of a person were recorded in the same session, and the simulated user always answers truthfully.
- **New words are simplified conventions** recorded by one person; their accuracy for other signers has not been measured. The DL spotter and the DL fallback know only the 13 original words.
- **Letters.** One signer, no stranger test, no "not a letter" class; `a`, `b`, `1` (and earlier `C`, `V`) are weak.

- **Not yet measured:** a live stranger test on the camera (tracked in `KIEM_TRA_TRUC_TIEP.md`), adversarial "Tier 5" tests, and the effect of different webcams at runtime.

4.11. Additional experiments after the first measurements

The measurements of Sections 4.2–4.8 were made on recorded sessions. After the first draft the system was exercised on the live camera and on degraded copies of the recordings. This section reports what these checks found. Unless stated otherwise, the numbers come from the owner's current bank (189 samples) and recordings, so they share the single-signer limitation of Section 4.10.

4.11.1. A defect that only the live camera could show

The recording tool mirrors every frame before it saves it, and the spotter was trained on these mirrored recordings. The live loop, however, handed the raw camera image to the spotter. All replay experiments used the recordings, so none of them went through this path. On the live camera the spotter probability stayed near zero and the indicator in the monitor panel did not move. The fault was reproduced offline by replaying a signed session horizontally flipped: the mean spotter probability on frames with a hand fell from 0.617 to 0.004, the 15 words normally committed fell to 0, and a negative session that normally commits no word let 3 stray words through.

Fix. The live camera is now mirrored by default (key `f` toggles it), and the landmarks given to the spotter are mirrored back whenever the displayed frame is not in the orientation of the training data. With the fix the flipped replay commits 14 words again and the negative session none, and the indicator moves on the live camera. **Lesson.** A component trained on recordings must be tested through the live input path, not only on replays. The letter classifier and the hand-shape gate normalise by the handedness label, which MediaPipe swaps on a mirrored image; on 18 frame pairs the label swapped in 15 and the feature vectors differed by a median of 0.29, so these two components are only approximately mirror-invariant. Mirroring the live camera by default removes the problem.

4.11.2. Spotter evaluation and augmentation

The spotter was evaluated out of fold (five folds grouped by recording attempt, 14 sessions): frame AUC 0.990, the gate is open on 90.5% of the signing frames, 0.7% of the stray-motion frames and 4.3% of the rest frames, and 97.3% of the signed words fall in an open gate. On a negative session recorded afterwards and never used for training (200 s of everyday movements), the unfiltered system committed 71 wrong words (21.3

per minute) and the system with the spotter committed 9 (2.7 per minute, mostly YES and DRINK).

The spotter turned out to be brittle to the geometry of the image. A second model was therefore trained with augmentation (mirroring, rotation ±12°, scaling 0.85–1.15, shifts of ±5% of the frame and landmark noise; three augmented copies of every session in the training folds). Table 20 and Figure 12 compare the two models on unperturbed, mirrored and randomly perturbed input of the held-out folds.

Table 20 – Spotter trained on originals versus trained with augmentation (out-of-fold)

Training	Test input	AUC	SIGN open	WANDER open	REST open	Words found	Spurious / min
Originals	Original	0.989	90.5%	0.7%	4.3%	97.3%	0.8
Originals	Mirrored	0.578	0.0%	0.5%	0.7%	0.0%	0.3
Originals	Perturbed	0.520	0.0%	0.4%	1.2%	0.0%	0.3
Augmented	Original	0.992	89.7%	0.7%	5.8%	91.9%	1.1
Augmented	Mirrored	0.994	90.7%	0.8%	5.8%	91.9%	1.1
Augmented	Perturbed	0.990	84.6%	1.0%	4.9%	81.1%	1.1

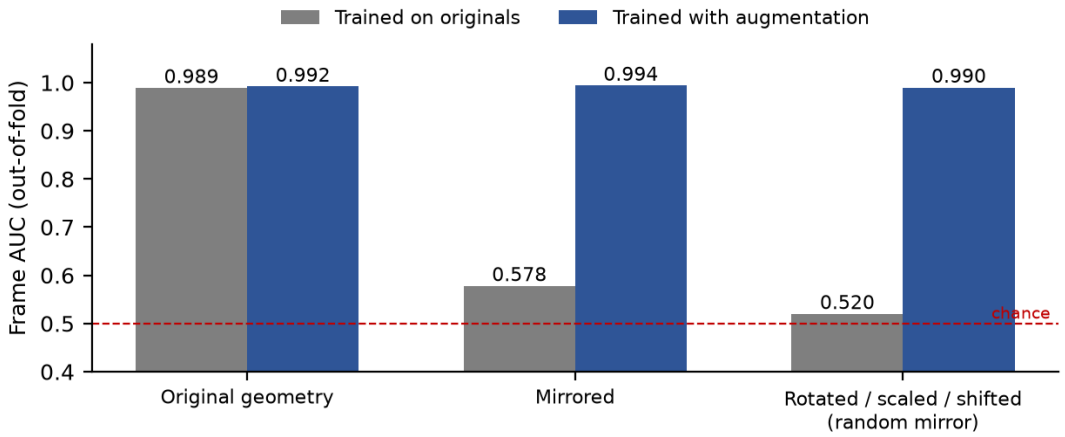


Figure 12 – Frame AUC of the spotter on original, mirrored and perturbed input, with and without augmentation

The model trained on originals falls to chance level (AUC 0.58 and 0.52) as soon as the image is mirrored or slightly perturbed, while the augmented model stays at 0.99. The price is a small loss on unperturbed data (words found 97.3% → 91.9%, spurious spans 0.8 → 1.1 per minute). The augmented model is a candidate only: it has not been evaluated on an independent session, so the production model is unchanged.

4.11.3. Robustness to image degradation

A 29-word signed session was degraded frame by frame and replayed with the default configuration (Table 21, Figure 13). Low resolution (one third), noise with blur, 15

frames per second and a subject standing farther away (image scaled to 60%) cost at most four words and add a few stray commits. **Over-exposure** (gain 1.5 plus an offset of 35, which saturates the skin) is the exception: only 6 of 29 words are recognised. On single images MediaPipe still found a hand in 47% of the frames instead of 62%, so the collapse is larger than the detection loss alone and also involves tracking and segmentation. Neither contrast-limited histogram equalisation nor a percentile stretch restored detection (40% and 33% against 47% unprocessed), so no pre-processing fix was adopted; the live loop already shows a "too bright" warning from its camera-health monitor, and the user is advised to avoid backlight.

Table 21 – Replay of one 29-word session under image degradations (default configuration)

Degradation	Committed	Correct	Duplicate	Wrong word	Stray	Missed
None (reference)	31	25	0	1	5	4
Over-exposure (×1.5 + 35)	7	6	0	1	0	23
Low resolution (÷3)	35	25	0	2	8	4
Noise and blur	33	24	1	2	6	5
15 frames per second	32	23	1	0	8	6
Subject far (60%)	31	21	1	2	7	8

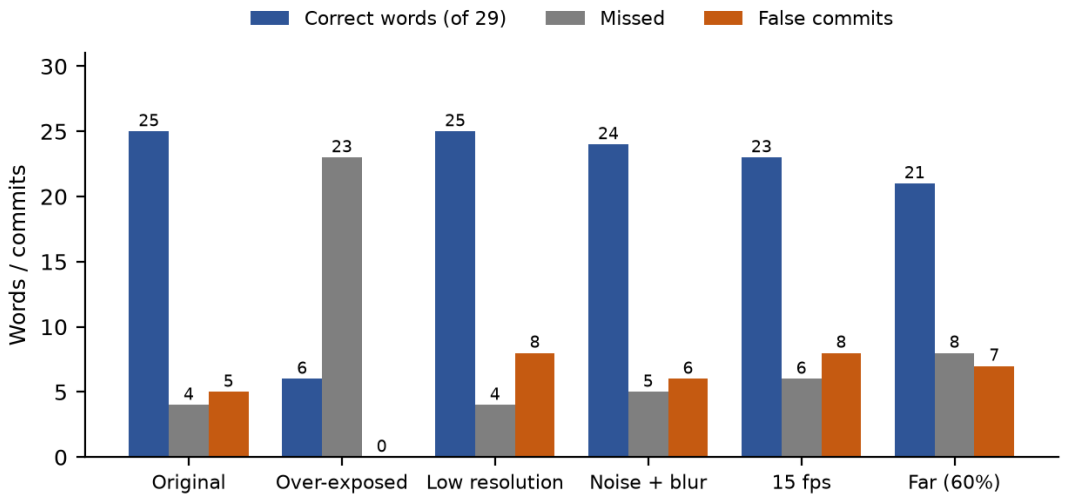


Figure 13 – Correct, missed and stray words when the recording is degraded before replay

4.11.4. The five added words

The five added words have no session identifier, so the leave-one-session-out protocol cannot query them. Instead the samples of every word were cut into four contiguous blocks and each block was queried against the others (demonstration bank of 18 words,

global-width configuration). The 13 original words keep 187 of 189 correct top-1 answers (99%); the new words obtain 36 of 47 (77%), with correct acceptance for 32 (Table 22, Figure 14).

Table 22 – Leave-block-out results of the five added words (18-word bank)

Word	Samples	Top-1 correct	Accepted correct	Accepted wrong
LIKE	8	8	8	0
WANT	8	7	6	1
MY	8	7	6	0
NAME	8	7	6	0
NO	15	7	6	2

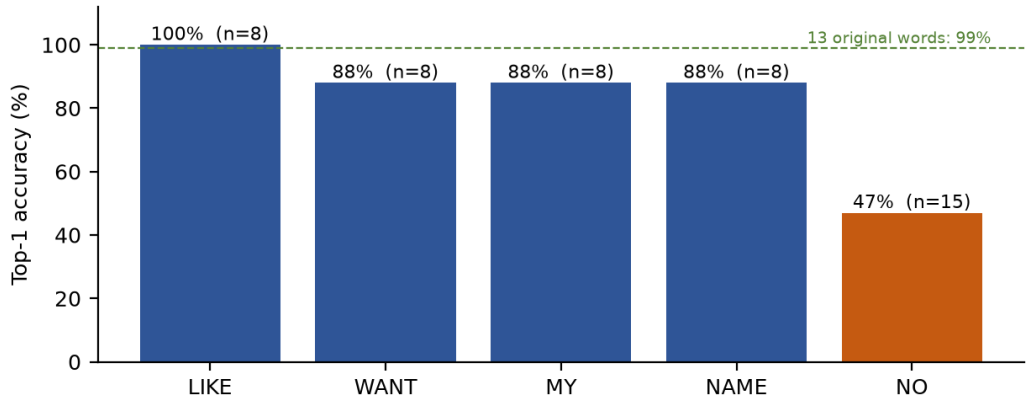


Figure 14 – Top-1 accuracy of the five added words (leave-block-out)

NO is the weak word: eight of its fifteen samples are labelled as another word (NAME twice, then YES, BYE, LIKE and DRINK), and a YES was accepted as NO once. These figures are optimistic: the blocks come from the same recording sitting, and the new words were recorded by a different person than the 13 original ones, so the matcher may separate them by signer rather than by gesture. They have not been measured on other signers.

4.11.5. Combining match scores for rejection: a negative result

The rejection score used so far is the ratio r between the best distance and the width of the winning word. We tested whether a logistic regression on several match quantities (the first three distances and ratios, the margin and the match quality) separates correct, accepted-worthy queries (358) from negatives (265: unknown words by leave-one-word-out, the 41 negative strokes and wrongly classified known words), with 6-fold cross-validation grouped by word or signer (Table 23).

Table 23 – Rejection scores compared (AUROC and recall of correct queries at 5% false acceptance)

Score	AUROC	Recall at 5% FPR
Ratio r (current)	0.894	0.726
Best distance d1	0.896	0.718
Margin between first and second	0.879	0.645
Logistic regression, 7 match features	0.898 – 0.902	0.746 – 0.763
… plus trajectory statistics	0.858 – 0.874	0.528 – 0.631

The best combination gains at most 0.008 AUROC, which is within the noise of this evaluation, and adding statistics of the query trajectory over-fits (AUROC falls to 0.86–0.87). The information needed to reject is not in these quantities, so the simple ratio is kept.

4.11.6. Collecting negative samples during normal use

Recording negative sessions is slow, and the spotter needs more of them. During recognition the key `n` now saves the last three seconds of landmarks as a short session labelled as stray motion (and removes a word committed in the same three seconds), and the key `x` removes the last word and logs it as wrong. The saved sessions are picked up by the next run of the spotter training command. The network is deliberately not updated online: a few samples from one person would shift it easily and no independent test would remain. This mechanism has been tested on recordings but its effect on the spotter has not been measured yet.

PART 5. CONCLUSIONS

5.1. Results achieved

- A complete real-time ASL pipeline on CPU: landmarks, segmentation, 24-D features, DTW with a three-level decision, sentence composition in English/Vietnamese, and non-blocking speech.
- A vocabulary of 13 production words and 18 demonstration words, 14 spelled letters, and a way to teach a word or adapt to a person with 1–3 samples without training.
- Measured performance: LOSO 100%; unseen signer 92.2% forced-choice top-1 and 0 of 175 unknown words accepted; 92–100% correct acceptance after enrolling 1–3 samples; 66–88% after a single session with confirmation questions; 96.6% with the optional gated DL fallback.
- A documented chain of replacements, each backed by measurement: image CNN → landmarks, shape label → 24-D features, searched weights → scoreboard weights, per-word → global threshold, frame-count → time-based segmentation, naive blend → gated fallback, pooling → enrolment.
- 485 passing automated tests and reproducible evaluation scripts with raw results.

5.2. Honest assessment

The system is **not** more accurate than a trained GRU when it only has to choose a word; its advantages are refusing unknown input, needing no training, being explainable and adapting within minutes. Fingerspelling and the new vocabulary are still weakly validated, and all findings rest on a small group of signers.

5.3. Future work

- Record more signers (8 or more) and re-measure with confidence intervals, including people outside the volunteer group.
- Extend the vocabulary and obtain proper two-hand tracking so that standard ASL signs can be used instead of simplified conventions.
- Retrain the spotter and the DL fallback on the new words; reconsider the GRU-agreement rescue rule with an independent threshold set.
- Improve fingerspelling: collect letters from several signers, add a "not a letter" class and fix `a`, `b`, `1`.
- Run the live stranger test and the adversarial test tier; verify `time_norm` on real webcams at other frame rates.

REFERENCES

[1] F. Zhang et al., "MediaPipe Hands: On-device Real-time Hand Tracking," arXiv:2006.10214, 2020. (Sections 2.3, 3.3)

[2] H. Sakoe and S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition," IEEE Trans. Acoustics, Speech, and Signal Processing, vol. 26, no. 1, pp. 43–49, 1978. (Section 2.5)

[3] M. Müller, Information Retrieval for Music and Motion, Springer, 2007, ch. 4 "Dynamic Time Warping". (Section 2.5)

[4] S. M. Pizer et al., "Adaptive histogram equalization and its variations," Computer Vision, Graphics, and Image Processing, vol. 39, no. 3, pp. 355–368, 1987. (Section 2.4)

[5] J. Canny, "A computational approach to edge detection," IEEE Trans. Pattern Analysis and Machine Intelligence, vol. 8, no. 6, pp. 679–698, 1986. (Section 2.4)

[6] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000. (Section 2.2)

[7] K. Cho et al., "Learning phrase representations using RNN encoder–decoder for statistical machine translation," EMNLP, 2014. (GRU baseline, Section 4.2)

[8] A. Desai et al., "ASL Citizen: A community-sourced dataset for advancing isolated sign language recognition," NeurIPS Datasets and Benchmarks, 2023. (Sections 2.1, 4.3.4)

[9] N. K. Caselli et al., "ASL-LEX: A lexical database of American Sign Language," Behavior Research Methods, vol. 49, pp. 784–801, 2017. (Section 4.7.3)

[10] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," NeurIPS, 2019. (Section 2.2)

[11] C. Tomasi and R. Manduchi, "Bilateral filtering for gray and color images," ICCV, 1998. (Section 2.4)

[12] G. Buchsbaum, "A spatial processor model for object colour perception," Journal of the Franklin Institute, vol. 310, no. 1, pp. 1–26, 1980. (Section 2.4)

[13] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," Psychometrika, vol. 12, pp. 153–157, 1947. (Section 2.6)

[14] Project documentation and raw results (folders docs/, specs/, results/ of the repository), cited in Sections 4.2–4.8.

APPENDIX A. THE PIPELINE OVERVIEW FILE AND THE SUBMITTED PACKAGE

A.1. pipeline.py: the main workflow in one short file

The file `pipeline.py` (about 270 lines) shows the whole workflow as eight calls, one per processing step, each documented with the method it applies (Figure 15, Table 24). It is a readable version of the production loop in `predict_sentence_webcam.py`: it uses the same components and the same configuration, but leaves out the user interface, speech, teaching, fingerspelling, early commit and lead trimming. On the first 150 s of a recorded session it produced the same sentences as the production loop except for one word (12 against 13 committed words).

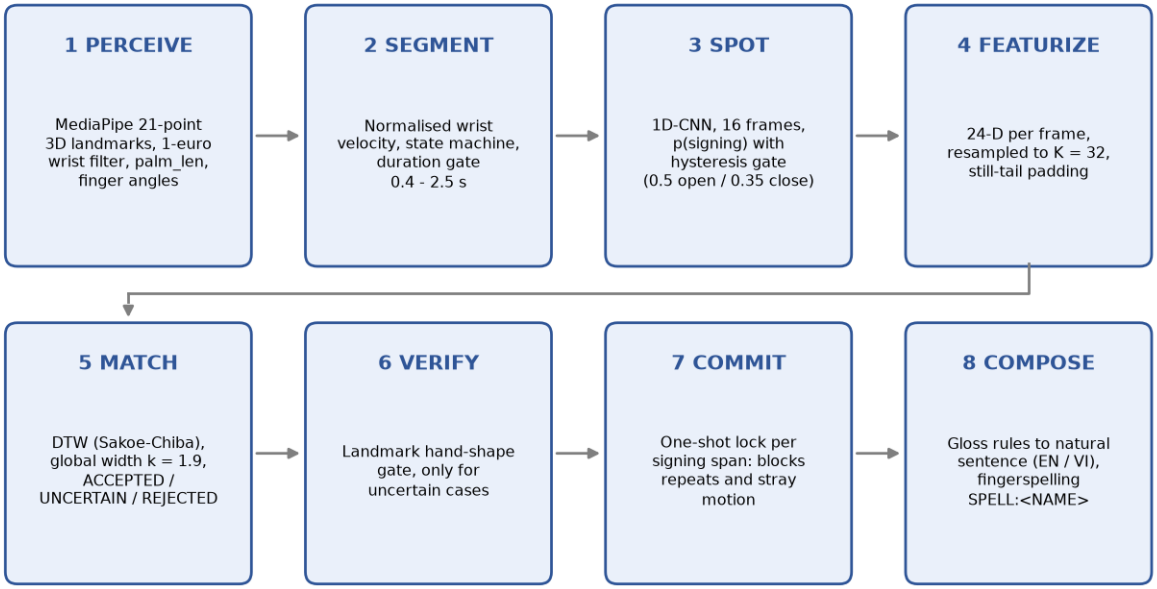


Figure 15 – The eight processing steps of pipeline.py

Table 24 – Steps of pipeline.py, the module that implements each and the method applied

Step	Implemented by	Method
1 PERCEIVE	ParallelHandDetector, compute_palm_len	MediaPipe landmarks, 1€ wrist filter, palm-length normalisation, finger flexion, palm normal
2 SEGMENT	KineticPauseSegmenter	Normalised wrist velocity, IDLE / RECORDING / PENDING_COMMIT, swipe left deletes
3 SPOT	DLSpotter, ProbGate	1D-CNN over 16 frames, dual-threshold gate
4 FEATURIZE	build_segment_features	24-D features, K = 32 resampling, duration and transition filters
5 MATCH	GoldenTriadDTWMatcher	DTW with Sakoe-Chiba band, weighted feature groups, width-based thresholds
6 VERIFY	HandshapeGateRuntime	Letter-landmark classifier and hand-shape table, only for uncertain results

Step	Implemented by	Method
7 COMMIT	OneShotLock	At most one word per signing span; blocks repeats and stray motion
8 COMPOSE	sentence_composer.compose	Gloss-to-sentence rules in English and Vietnamese

A.2. Submitted package

The submitted folder `DIP_nop_bai` contains only what is needed to run the software: 49 Python files, the trained models, the template banks, the web front end, the audio cache, a 45-second demonstration clip and the tests that apply to the included modules. To run it:

```
py -m pip install -r requirements.txt
py dip.py                                    (window application)
py dip.py nhan-dien                          (live recognition)
py pipeline.py --video demo/demo_clip.mp4    (workflow on the demonstration clip)
```

Not included are the raw recordings, the ASL Citizen videos and the experiment outputs; the evaluation commands of `dip.py` that need them cannot be reproduced from the package alone.

APPENDIX B. MAIN SOURCE FILES

Table 25 lists the files that carry the method, with their approximate length. All of them are in the root folder of the submitted package.

Table 25 – Main source files and their role

File	Lines	Role
pipeline.py	270	The whole workflow in eight calls (Appendix A)
predict_sentence_webcam.py	3100	Full live loop: interface, speech, teaching, fingerspelling, event log
dynamic_dtw_engine.py	2600	KineticPauseSegmenter and GoldenTriadDTWMatcher (DTW and three-level decision)
dynamic_features.py	550	24-D trajectory features, palm length, resampling, face-box landmark
pipeline_common.py	615	Parallel hand detector, wrist filter, segmenter factory
improved_dip_pipeline.py	990	Image-processing chain: bilateral filter, skin segmentation, moments, Canny, Sobel
spotter_dl.py, spotter_features.py, spotter_gate.py	500	Spotter network, its frame features, hysteresis gate and one-shot lock
handshape_gate.py, hand_pose_features.py, letter_landmark_classifier.py	450	Landmark hand-shape classifier and gate
sentence_composer.py, fingerspell.py	530	Gloss-to-sentence rules and fingerspelling
session_enroll.py, online_negatives.py	350	Teaching a word and collecting negative samples
dip.py, dip_app.py	1270	One-command launcher and window application
scoreboard.py, eval_*.py	3700	Evaluation protocols (leave-one-session-out, unseen signer, open set)